PulseStream

Near-Real-Time Event Analytics Pipeline · with Embedded Anomaly Detection

High-Level Design · solo build · mid-level difficulty · ~4–8 weeks of evenings

*This is a design document, not a tutorial. It gives architecture, component responsibilities, key decisions/trade-offs, and a phased roadmap — deliberately without finished code. Spots marked **DECIDE** are open questions intended for you to resolve as part of the build.*

1 Why this project

The 2026 data landscape has shifted from batch-first to streaming-native designs, and the "data wrangling" part is increasingly commoditized by AI assistance. The differentiator now is the **engineering**: reliability, decoupling, testing, observability, and cost. This project is chosen so that the hard, interesting work is exactly that — while staying genuinely data-oriented. It is a strong portfolio piece because it is a complete, operable system rather than a notebook.

What it exercises, end to end

- Real-time ingestion from a live source (reconnection, backpressure, ordering)
- Incremental stream processing and windowed feature computation
- A lightweight statistical / ML inference step embedded inside the pipeline
- An analytical storage layer (columnar, query-on-files) plus a typed API and a dashboard
- Containerization and a full CI/CD pipeline with automated deploy
- Deterministic tests for a non-deterministic system, plus production-style observability

2 Language & staying on trend

Primary language: Python. It is still the default for data work in 2026 (it recently overtook JavaScript as the most-used language on the Stack Overflow survey), so it is the right call given your background. "Staying on trend" here does *not* mean switching languages — it means using the modern, performance-backed tooling:

- **Polars** (Rust core) for heavier batch transforms & backfills
- **DuckDB** (C++ core) as the embedded analytical engine
- **uv** for fast, lockfile-based dependency & environment management
- **Ruff** for lint + format (one fast tool), **mypy** for type-checking
- **Pydantic v2** for typed events & settings, **FastAPI** for the async typed API

Optional language stretch (only if you want it)

Write *one* hot-path component (e.g. the event parser/validator or a rolling aggregator) as a small Rust extension via PyO3/maturin, and benchmark it against the Python version. Go is also popular for streaming/infra services, but rewriting in Go would slow you down with little payoff for a solo data project — skip it unless a Go service is itself a learning goal.

3 System architecture

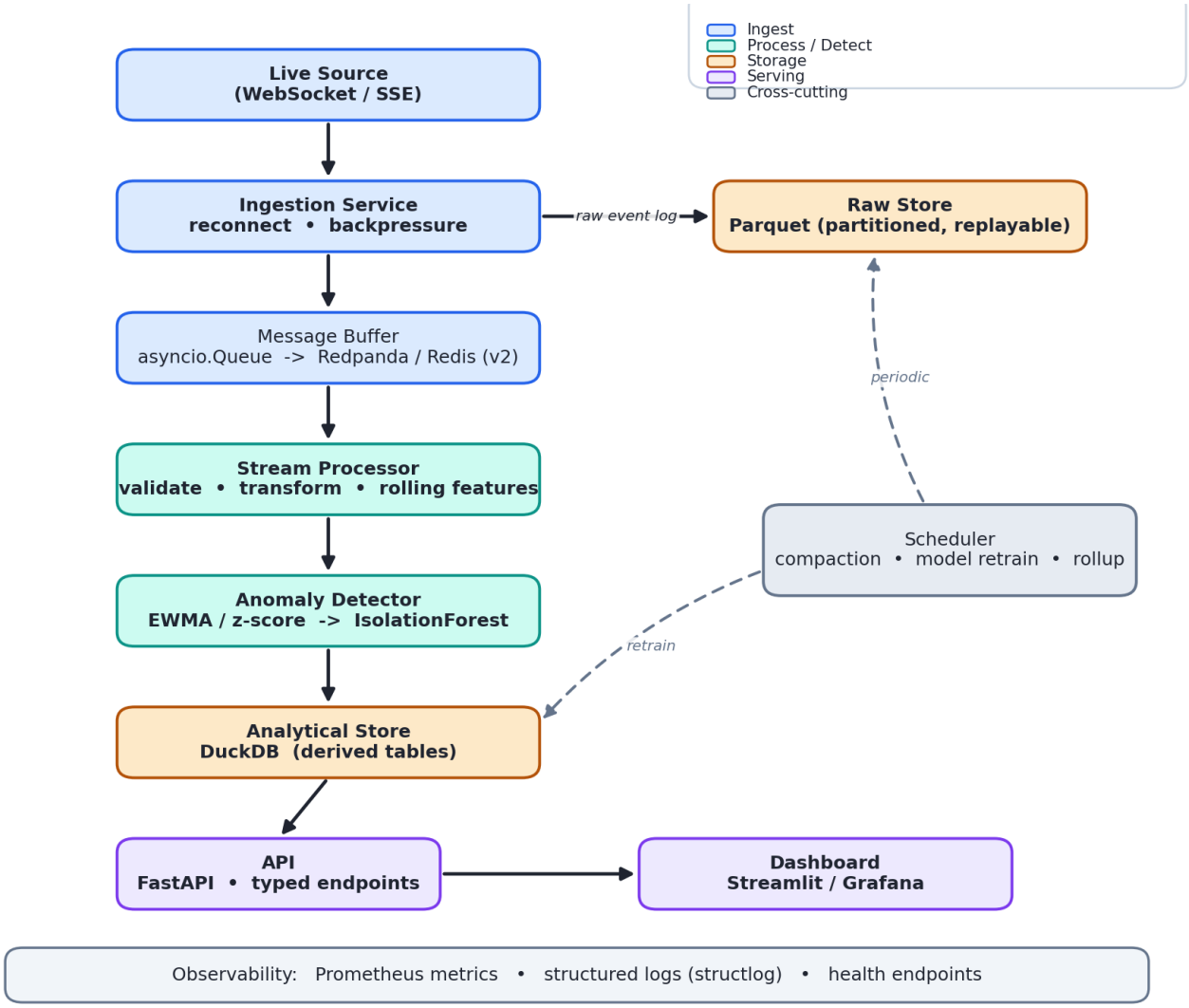


Figure 1 — PulseStream system architecture. Solid arrows are the main event flow; dashed arrows are periodic scheduler jobs. Colors group components by layer.

Events flow from the live source through ingestion into a buffer, are validated and turned into rolling features by the processor, scored by the anomaly detector, and landed in DuckDB for fast querying by the API and dashboard. Raw events are also appended to partitioned Parquet so the whole pipeline is replayable.

Phasing note — the message buffer

v1: skip a broker entirely. Use an in-process `asyncio.Queue` so ingestion and processing run in one service and you reach a working end-to-end slice fast. **v2:** introduce a real broker (Redpanda — Kafka-compatible single binary — or Redis Streams) to decouple ingestion from processing into separate services, with consumer groups and replay. Doing it in this order is deliberate, defensible engineering.

4 Goals & non-goals

Goals

- Continuously ingest a live event stream with no data loss under normal operation
- Transform raw events into clean, typed records and rolling features
- Detect anomalies in near-real-time and surface them via API + dashboard
- Persist both raw events (replayable) and derived aggregates (queryable)
- Run the whole system locally with one command; ship a full CI/CD pipeline
- Be observable: metrics, structured logs, health endpoints

Non-goals (resist scope creep)

- Horizontal scale / multi-node clusters — single-node is fine and on-trend
- Exactly-once delivery — aim for at-least-once + idempotent writes instead
- A designer-grade frontend — a functional dashboard is enough
- Multi-tenant auth/users — a single API key (or open local API) is fine for v1
- A general-purpose framework — build for one data source first

5 Recommended data sources (pick one to start)

You want a free, genuinely real-time, high-enough-volume source. Put it behind an `EventSource` interface so you can swap or add sources later.

Option	Source	Why / anomaly signal
A	Crypto exchange public WebSocket (Coinbase / Binance)	Engaging, truly real-time, no auth for public market data. Anomalies = price/volume spikes. High demo factor.
B	Wikimedia EventStreams (SSE)	Completely free, no auth, no rate limits, very robust, high volume. Anomalies = edit-rate spikes, bot surges, large byte-deltas.
C	Public transit GTFS-Realtime (vehicle positions)	Spatial + temporal. Anomalies = bunching, gaps, stalled vehicles. More setup.

Recommendation: start with **B (Wikimedia)** for a frictionless first end-to-end slice, or **A (crypto)** if market data motivates you more. **DECIDE:** which source, and what your headline “anomaly” means for it.

6 Tech stack

Concern	Choice	Why
Language / env	Python 3.12+, uv	Default for data; fast lockfile-based env management
Lint / format / types	Ruff, mypy	One fast tool for lint+format; catch interface bugs early
Validation / config	Pydantic v2	Typed events and typed settings
Async I/O	asyncio + websockets / httpx-sse	Right model for stream ingestion
Transforms	Python; Polars for batch	Polars (Rust) for heavier backfills
Broker (v2)	Redpanda or Redis Streams	Kafka-compatible / simple to run; decoupling + replay
Analytical store	DuckDB	Embedded OLAP, on-trend, zero ops

Concern	Choice	Why
Raw store	Parquet (partitioned)	Columnar, replayable, queryable directly by DuckDB
Anomaly detection	stats first, then scikit-learn	EWMA/z-score/MAD baseline; IsolationForest as v2
API	FastAPI + uvicorn	Async, typed, auto OpenAPI docs
Dashboard	Streamlit or Grafana	Streamlit = fast to build; Grafana = ops-realistic
Scheduler	APScheduler → Prefect/Dagster	Periodic jobs; workflow tooling as a stretch
Logging / metrics	structlog, prometheus-client	Structured logs; rates, lag, anomalies, errors
Tests	pytest, Pandera/GE	Deterministic via replay; data-quality gates
Containers / CI/CD	Docker, GitHub Actions, GHCR	One-command local; lint+test+build+scan+deploy

7 Component breakdown

Ingestion service. Maintains a durable connection (reconnect with exponential backoff + jitter), normalizes each message into a raw envelope, appends it to the Parquet raw store, and hands it to the buffer. **DECIDE:** backpressure policy (block vs. drop-oldest).

Message buffer. v1 in-process `asyncio.Queue`; v2 a Redpanda topic or Redis Stream with consumer groups, offsets, and replay. **DECIDE (v2):** partition key and retention.

Stream processor. Consumes raw events, validates into typed domain events (Pydantic), enriches, and maintains rolling windows of features (rate, EWMA, rolling mean/std, byte-delta/volume). **DECIDE:** window size(s) and tumbling vs. sliding.

Anomaly detector. v1 statistical (z-score / MAD / EWMA control limits / thresholds) emitting an Anomaly record. v2 model-based (IsolationForest) with periodic retraining and versioned, atomically-swapped artifacts.

Analytical + raw store. Parquet raw = immutable source of truth (enables replay/backfill); DuckDB holds derived tables (events_enriched, features_rolling, anomalies, agg_minutely) and can query Parquet directly. A scheduled job compacts small files. **DECIDE:** micro-batch cadence (recommended over per-event).

API layer. FastAPI with Pydantic response models: /health, /metrics, /stats/latest, /timeseries, /anomalies, and (stretch) a live /anomalies/stream over SSE/WebSocket.

Dashboard. Streamlit (fast) or Grafana on the store. Read *through the API* where practical, not straight from the DB — a cleaner boundary and a better engineering story.

Scheduler. APScheduler worker for compaction, model retrain, and retention/rollup; optionally Prefect/Dagster as a stretch for retries, scheduling, and run observability.

8 Data model (sketch — refine to your source)

```
RawEvent (envelope)
  ingest_id, source, received_at (UTC), schema_version, raw_payload

DomainEvent (validated, source-specific)
  event_id, entity, event_ts (UTC), <numeric fields...>, derived...

FeatureSnapshot
  entity, window_start, window_end, rate, ewma, mean, std, ...

Anomaly
  anomaly_id, detected_at, entity, metric, value, score, method,
  severity, model_version (nullable)

AggMinutely (fast charts)
  bucket_ts, entity, count, avg_value, max_value, anomaly_count
```

Conventions: all timestamps UTC and timezone-aware. Store *event_ts* (when it happened) separately from *received_at* (when you got it) — you need both to reason about lateness and lag.

9 CI/CD pipeline (GitHub Actions)

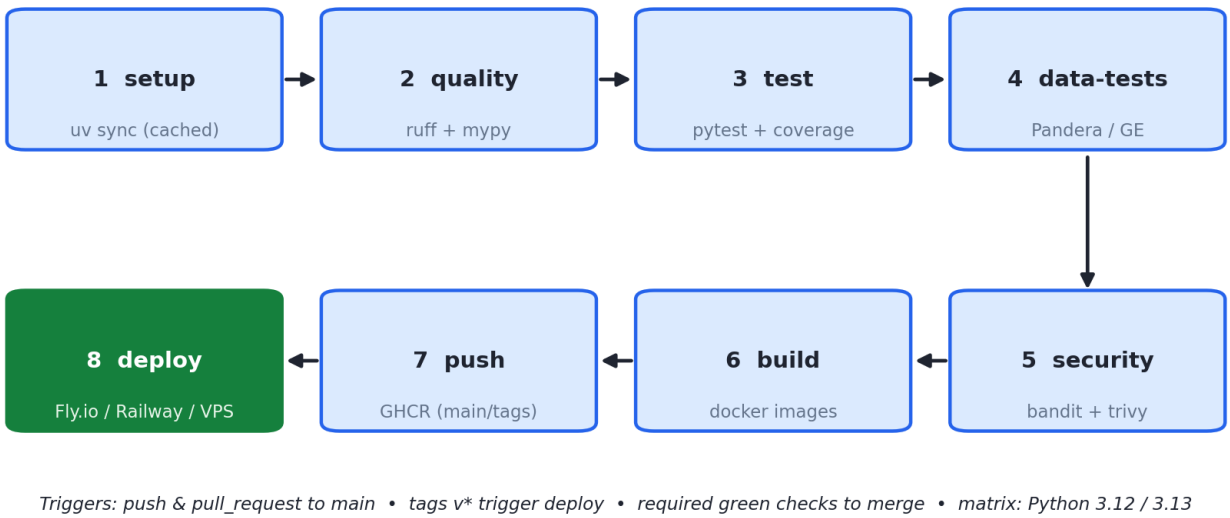


Figure 2 — CI runs on every push/PR; tagged releases (v) trigger the deploy stage.*

Good practices to demonstrate

- Cache the uv env and Docker layers so CI stays under a couple of minutes
- Required status checks on main (no merge unless green); matrix across Python 3.12 / 3.13
- Pre-commit hooks (ruff, mypy, whitespace) run the same checks locally as CI does
- GitHub Environments + secrets for deploy creds — never hardcode them
- Image security scan (trivy) and code scan (bandit) as part of the gate

DECIDE: deploy target. Fly.io and Railway are the least-friction and give a public URL; a cheap VPS via SSH + Compose is the most realistic and teaches the most ops. Either one completes the CI/CD loop.

10 Testing strategy

The challenge is a non-deterministic live source. Solve it with **replay**: capture a few minutes of real events once, save them as a fixture, and provide a `FakeSource` that yields them (at adjustable speed). All tests run against FakeSource — no live network in CI.

- **Unit:** transforms, feature math, anomaly scoring (pure functions — easy wins)
- **Integration:** FakeSource → processor → DuckDB; assert rows and aggregates
- **Anomaly eval:** inject *synthetic* anomalies into the replay; assert they are flagged; measure precision/recall
- **API / contract:** drive the ASGI app with httpx; assert responses match the Pydantic/OpenAPI contract
- **Data quality:** Pandera or Great Expectations schema checks (types, ranges, non-null) as a gate

11 Observability & monitoring

Expose Prometheus metrics at `/metrics` on each service:

- `ingest_events_total` (counter) and `errors_total` by stage
- `processing_lag_seconds` = now – event_ts of last processed event
- `queue_depth` / consumer lag, and `anomalies_detected_total` by severity
- `processing_duration_seconds` (histogram)

Logs: structured JSON via structlog, always carrying ingest_id/event_id so you can trace an event across services. Health: `/health` distinguishing liveness (process up) from readiness (source connected, store reachable). **Stretch:** a Prometheus + Grafana container in compose.

12 Anomaly detection, illustrated

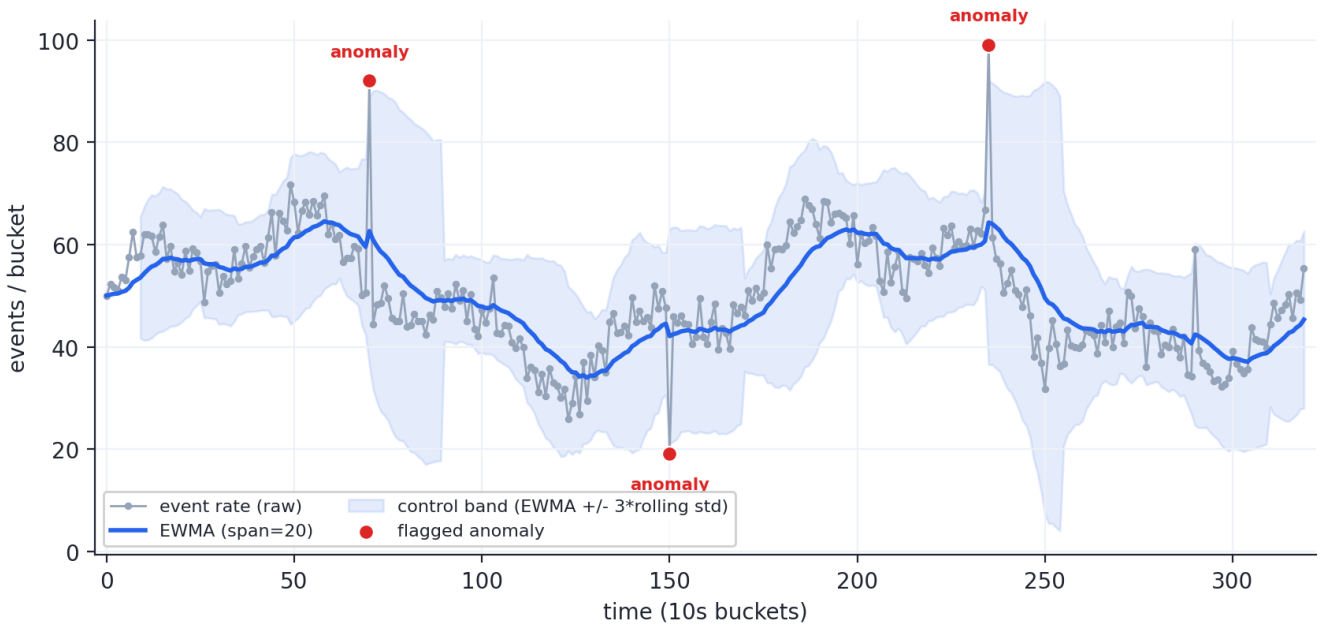


Figure 3 — v1 statistical detector: an EWMA control band over the event-rate stream. Points outside EWMA ± k·(rolling std) are flagged. Note the missed spike inside a widened band — a real limitation worth measuring.

Start with statistics, not a model: EWMA control limits and MAD-based z-scores are robust, explainable, and a sane baseline. v2 adds a scikit-learn IsolationForest trained periodically on a window of historical features pulled from DuckDB. **DECIDE:** with no labels, evaluate detector quality by injecting synthetic anomalies into a replayed stream and measuring recall/precision — that is your detector's test.

13 Phased roadmap

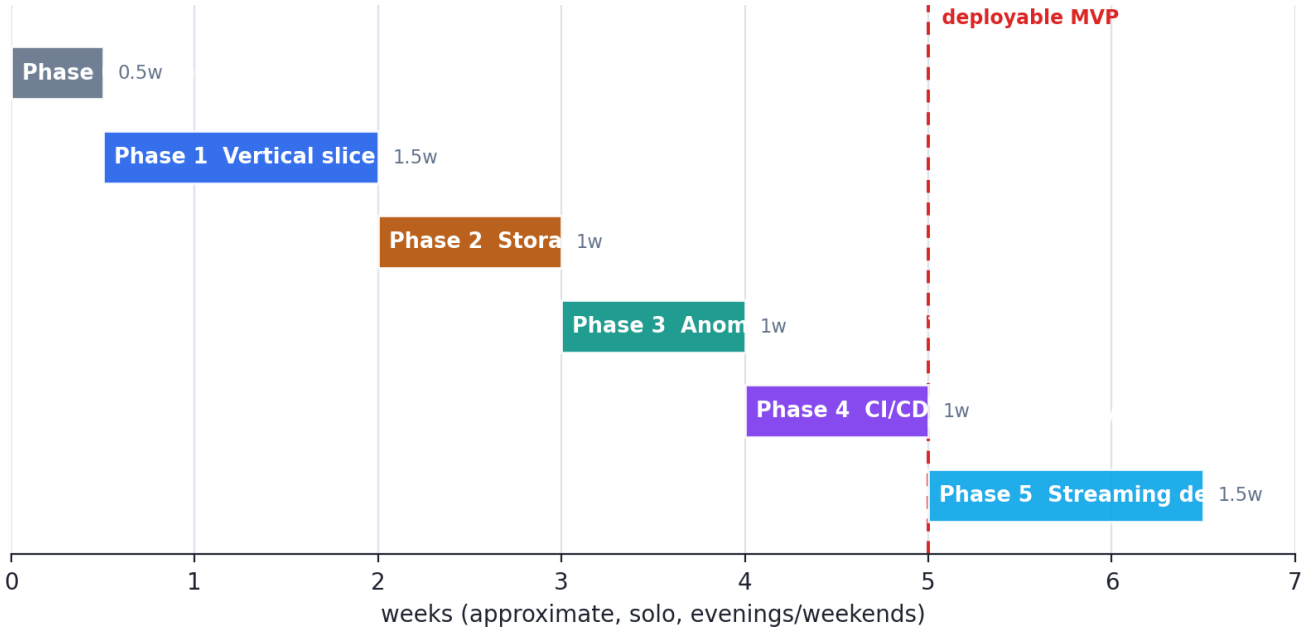


Figure 4 — each phase is an independently demoable increment; a deployable MVP exists by the end of Phase 4, before the "real" streaming work in Phase 5.

- **Phase 0 — Skeleton:** repo, uv, Ruff/mypy/pytest, pre-commit, a green CI on an empty app
- **Phase 1 — Vertical slice:** one source → in-process queue → validate → DuckDB → one API endpoint, tested via FakeSource
- **Phase 2 — Storage + analytics:** partitioned Parquet + compaction, DuckDB derived tables, /timeseries, a Streamlit chart
- **Phase 3 — Anomaly detection v1:** rolling features + statistical detector, anomalies table/endpoint/panel, synthetic-anomaly eval
- **Phase 4 — CI/CD + observability:** full pipeline, deploy on tag, Prometheus metrics + health, public URL
- **Phase 5 — Streaming decoupling:** introduce Redpanda/Redis, split ingestion and processing, consumer groups + replay

14 Stretch goals

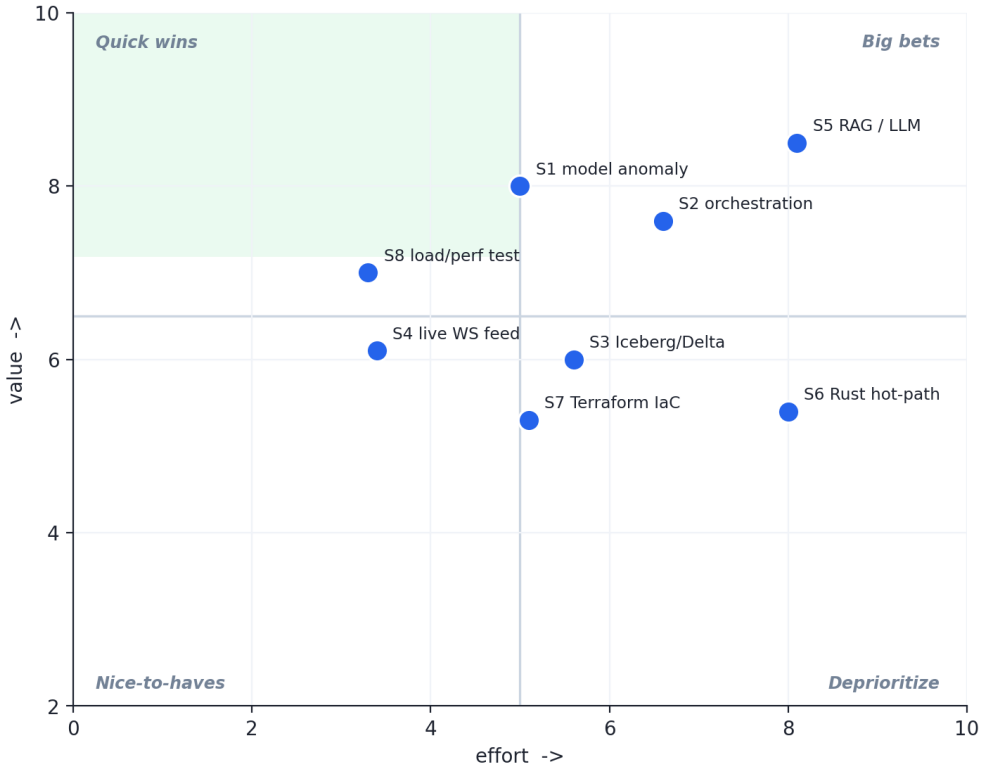


Figure 5 — stretch goals by value vs. effort. Prefer the upper-left quadrant.

- **S1** model-based anomaly detection (IsolationForest) with scheduled retraining & versioned artifacts
- **S2** workflow orchestration with Prefect/Dagster (retries, scheduling, run UI) — very on-trend
- **S3** open table format (Apache Iceberg / Delta) for ACID + time-travel on the lake
- **S4** live anomaly feed over WebSocket/SSE (push, not poll)
- **S5** a RAG/LLM angle: index recent events/anomalies into a vector store + a natural-language /ask endpoint
- **S6** a Rust hot-path component via PyO3/maturin, built in CI and benchmarked
- **S7** infrastructure-as-code (Terraform) for the deploy target
- **S8** load/perf test (replay at high speed) with a documented throughput/lag profile

15 Key design decisions & trade-offs

#	Decision — and the trade-off to be ready to explain
D1	**In-process queue first, broker later.** Less “real” early, but you reach a working system fast and isolate broker complexity to one milestone.
D2	**At-least-once + idempotent writes, not exactly-once.** Exactly-once is hard and rarely worth it solo; dedupe on event_id is simpler and sufficient.
D3	**DuckDB (single-node embedded) over a server warehouse.** No concurrency/HA, but zero ops and excellent analytics — correct for scope (breaks at true concurrency / >~500GB).
D4	**Micro-batch writes over per-event writes.** Slightly higher latency for big columnar/file efficiency gains.

#	Decision — and the trade-off to be ready to explain
D5	**Dashboard reads through the API, not the DB.** A little more work for a cleaner boundary and a better engineering story.
D6	**Statistics before ML for anomalies.** Less flashy, but robust, explainable, and a baseline to measure any model against.

16 Suggested repo structure

```
pulsestream/
  pyproject.toml            # uv-managed; Ruff/mypy/pytest config
  uv.lock
  .pre-commit-config.yaml
  .env.example
  docker-compose.yml
  Makefile                  # run / test / lint / up / down / seed
  .github/workflows/ci.yml
  docs/   hld.pdf  decisions/   # short ADRs for D1..D6
  src/pulsestream/
    common/      # settings, logging, Pydantic models, metrics
    sources/     # EventSource protocol + Wikimedia/Crypto/Fake
    ingestion/   processing/   detection/
    storage/     # parquet writer, duckdb access, compaction
    api/         dashboard/    scheduler/
  tests/
    fixtures/    # captured event replay file(s)
    unit/  integration/  api/  data/
  docker/  Dockerfile.<service>   # multi-stage, non-root
```

First three concrete steps

1. Pick your data source (§5) and define what “anomaly” means for it.
2. Do Phase 0: scaffold the repo with uv + Ruff + mypy + pytest and a green GitHub Actions run on an empty app — get the pipeline working before any logic.
3. Build the Phase 1 vertical slice end-to-end (source → queue → validate → DuckDB → one endpoint), tested against a FakeSource.